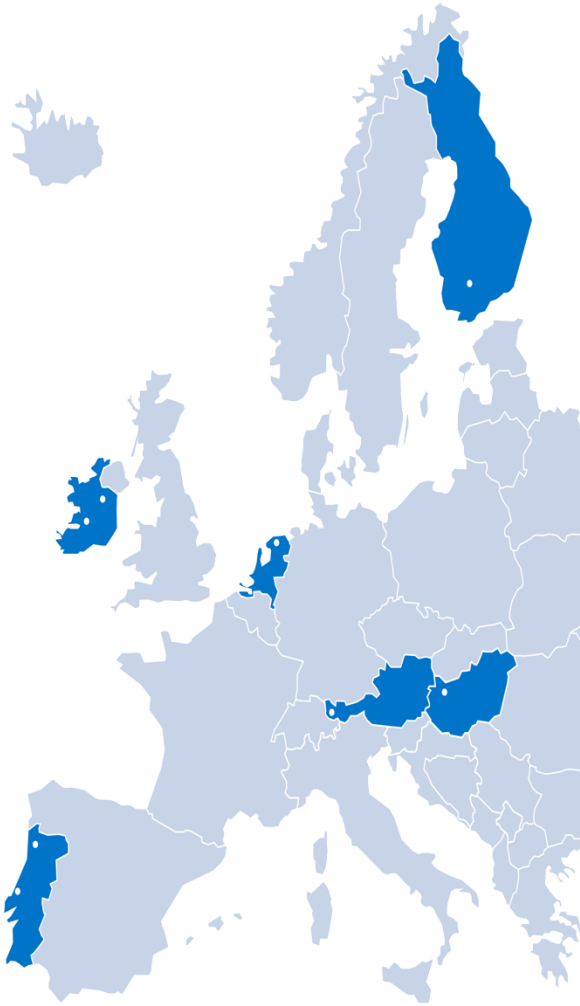
REGIONAL UNIVERSITY NETWORK

EUROPEAN UNIVERSITY



CRYPTO: SIGNATURES, HASHES

BIP HACK YOUR DEVICE 2026

ARMIN SIMMA

Co-funded by the Erasmus+ Programme of the European Union

The content of this publication represents the views of the author only and is his/her sole responsibility. The European Commission and the Agency do not accept any responsibility for use that may be made of yhe information it contains. Grant Agreement Number: 101004068.-

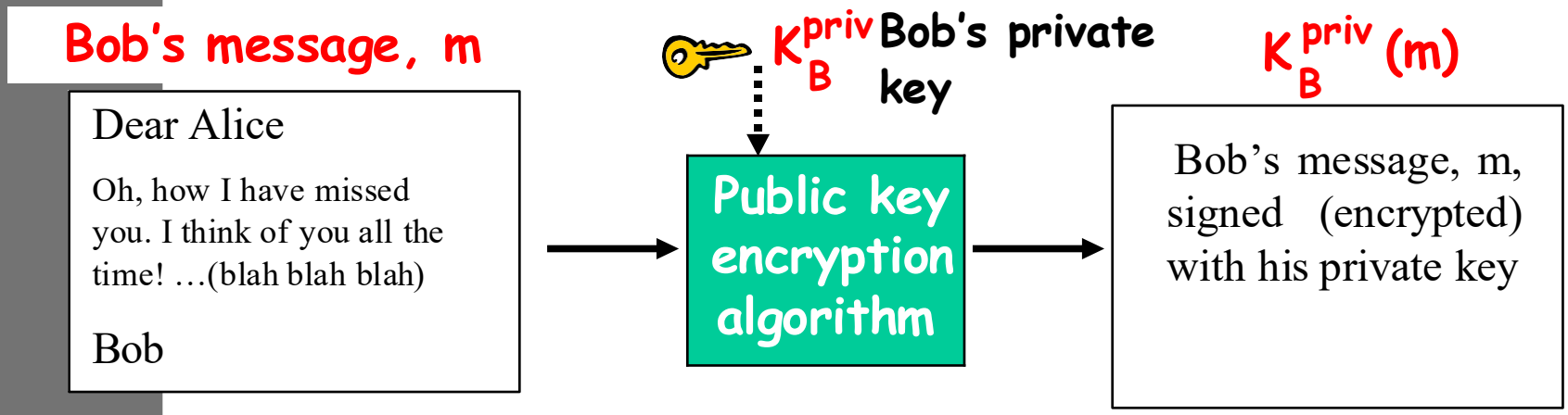
POLITÉCNICO DE LEIRIA · TUS · IPCA INSTITUTO POLITÉCNICO DO CÁVADO E DO AVE · HAMK · SZÉCHENYI EGYETEM · NHL STENDEN · FHV Vorarlberg University of Applied Sciences

Cryptographic technique analogous to hand-written signatures.

- sender (Bob) digitally signs document, establishing he is document owner/creator.
- verifiable, nonforgeable, nonrepudiable: recipient (Alice) can prove to someone that Bob, and no one else (including Alice), must have signed document

Simple "digital signature" for message m:

- Bob signs m by encrypting with his private key K_B,
- creating "signed" message, $K_B(m)$



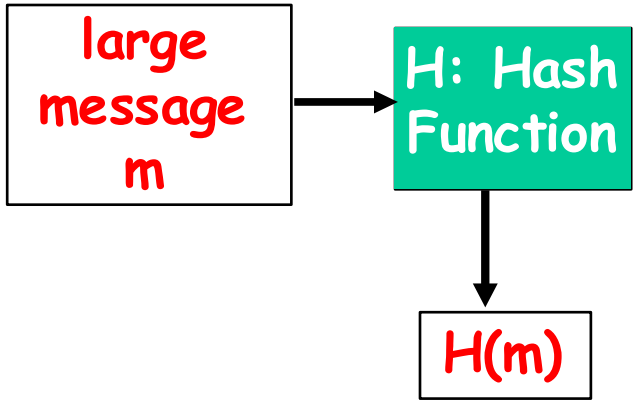
For exam: Not the real digital signature!

Message Digests (Hashes)

Computationally expensive to public-key-encrypt long messages

Goal: fixed-length, easy- to-compute digital "fingerprint"

- apply hash function H to *m*, get fixed size message digest, *H(m)*.



Hashing

- Hashing is a one-way function. It cannot be reversed
  - From the hash, you cannot compute the original message
- Hashing is repeatable
  - If two parties apply the same hashing method to the same bit string, they will get the same hash
  - Hashing must be collision-resistant
  - Details see next slides (hidden)

Figure 7-12: Encryption Versus Hashing

	Encryption	Hashing
Use of Key	Uses a key as an input to an encryption method	Key is not necessary. if Key: is usually added to text; the two are combined, and the combination is hashed
Length of Result	Output is similar in length to input	Output is of a fixed short length, regardless of input
Reversibility	Reversible; ciphertext can be decrypted back to plaintext	One-way function; hash cannot be "de-hashed" back to the original string

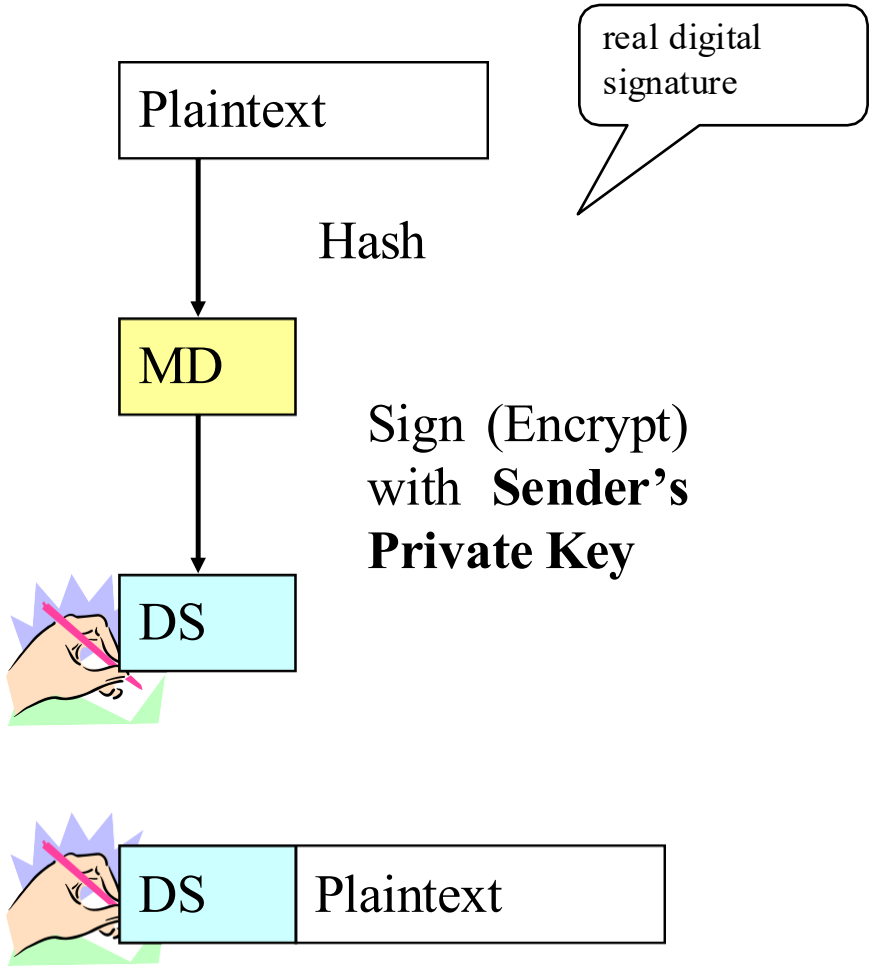
Digital Signature for Message-by-Message Authentication

To Create the Digital Signature:

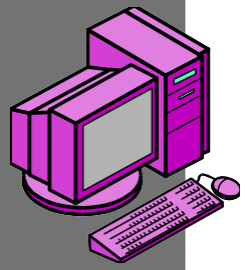
1. **Hash** the plaintext to create a brief message digest;; this is NOT the Digital Signature.

2. **Sign** (encrypt) the message digest with the sender's private key to create the digital signature.

3. Transmit the plaintext + digital signature, encrypted with symmetric key encryption.

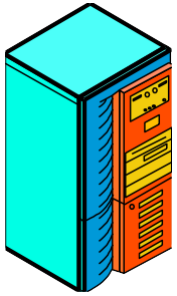


Digital Signature for Message-by-Message Authentication



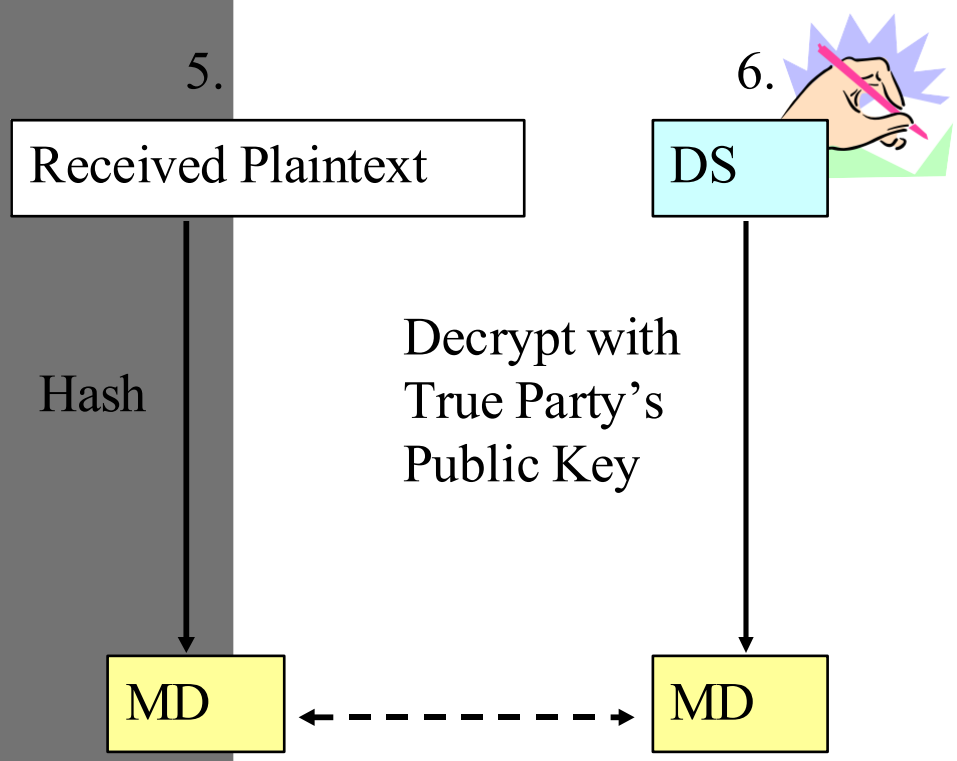
Sender

4. Encrypted with Session Key



Receiver

Digital Signature for Message-by-Message Authentication



To Test the Digital Signature

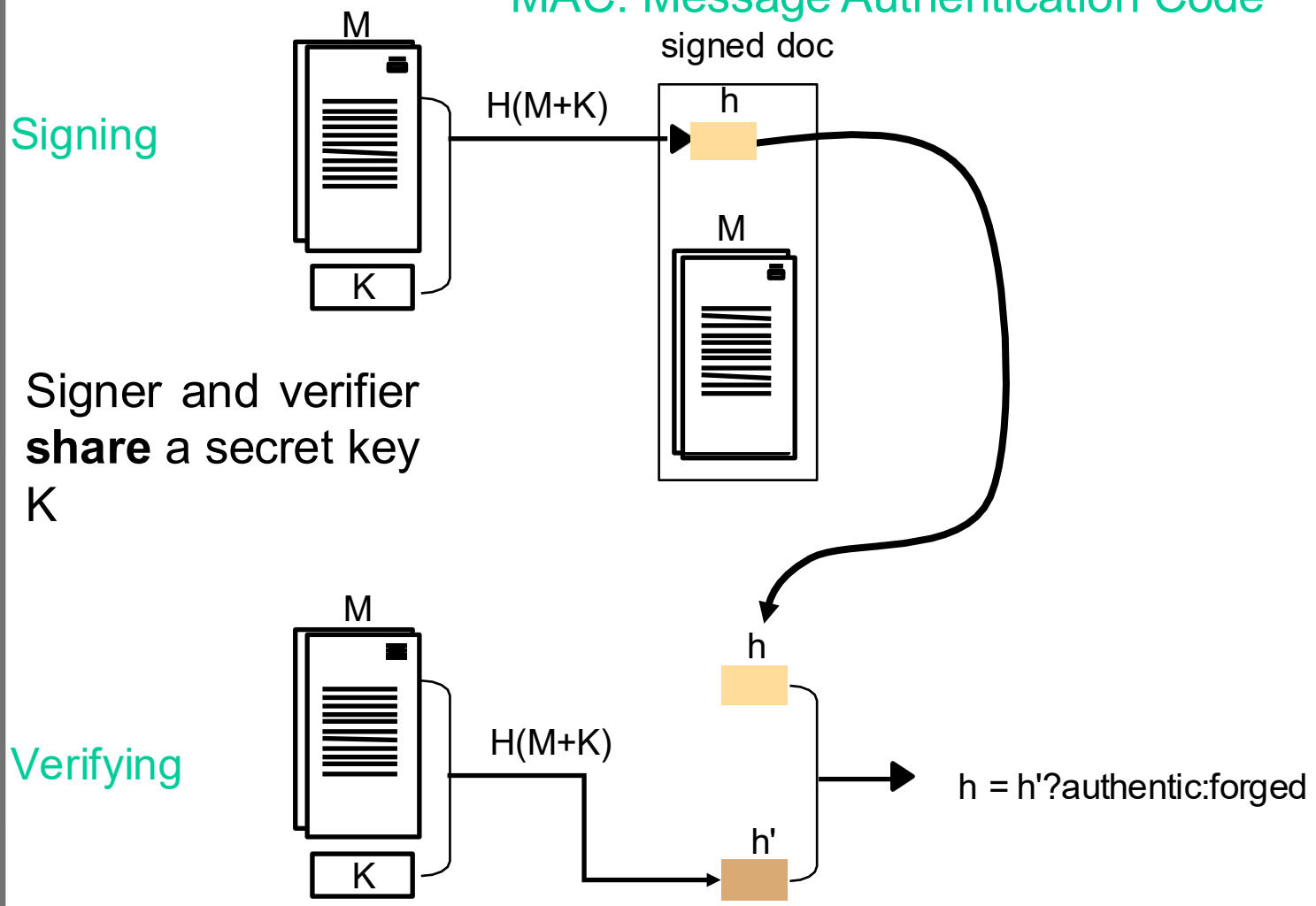
5. Hash the received plaintext with the same hashing algorithm the sender used. This gives the message digest.

6. Decrypt the digital signature with the sender's public key. This also should give the message digest.

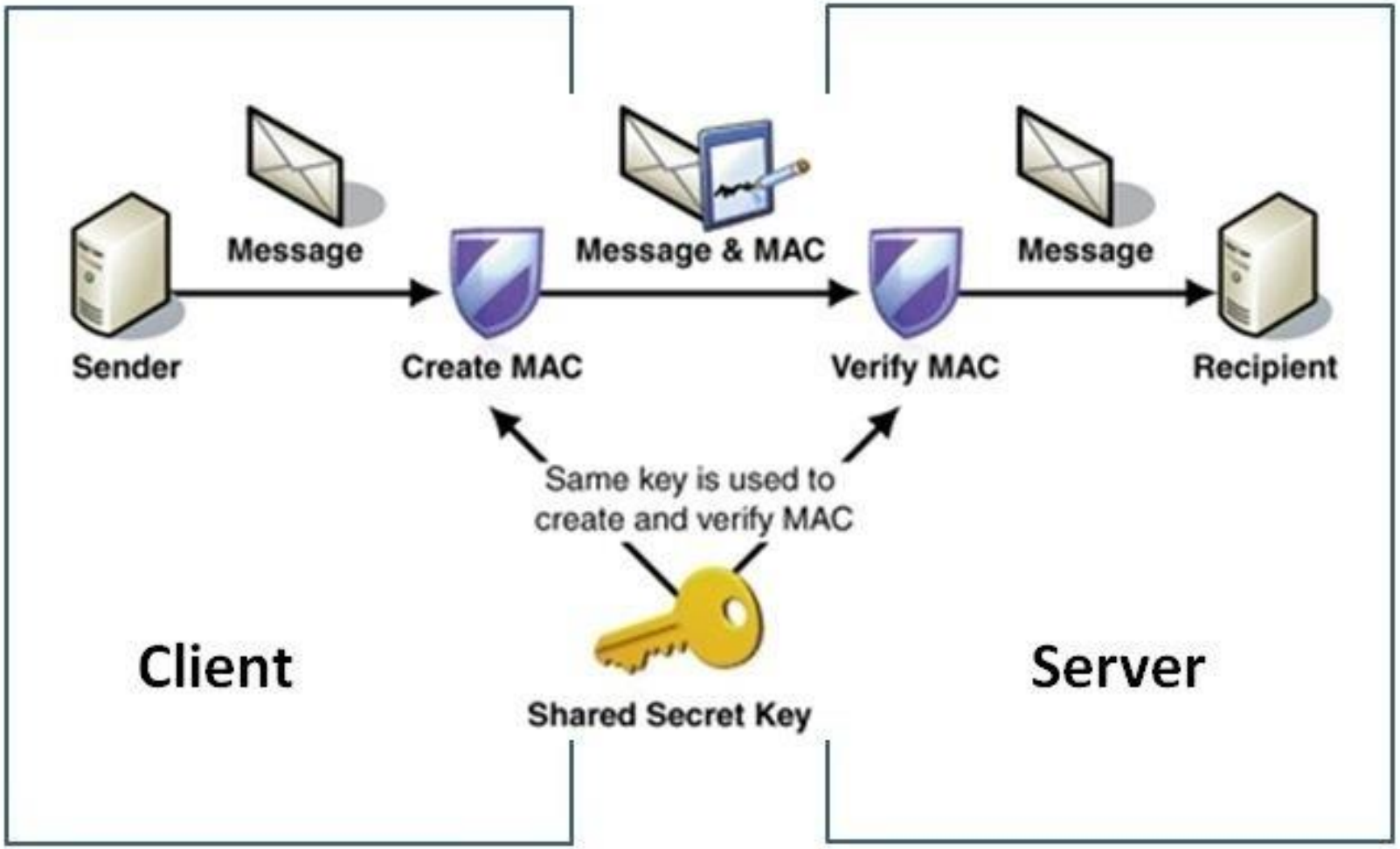
7. If the two match, the message is authenticated.

MACs: Low-cost signatures with a shared symmetrical secret key

MAC: Message Authentication Code



MAC Message Authentication Code



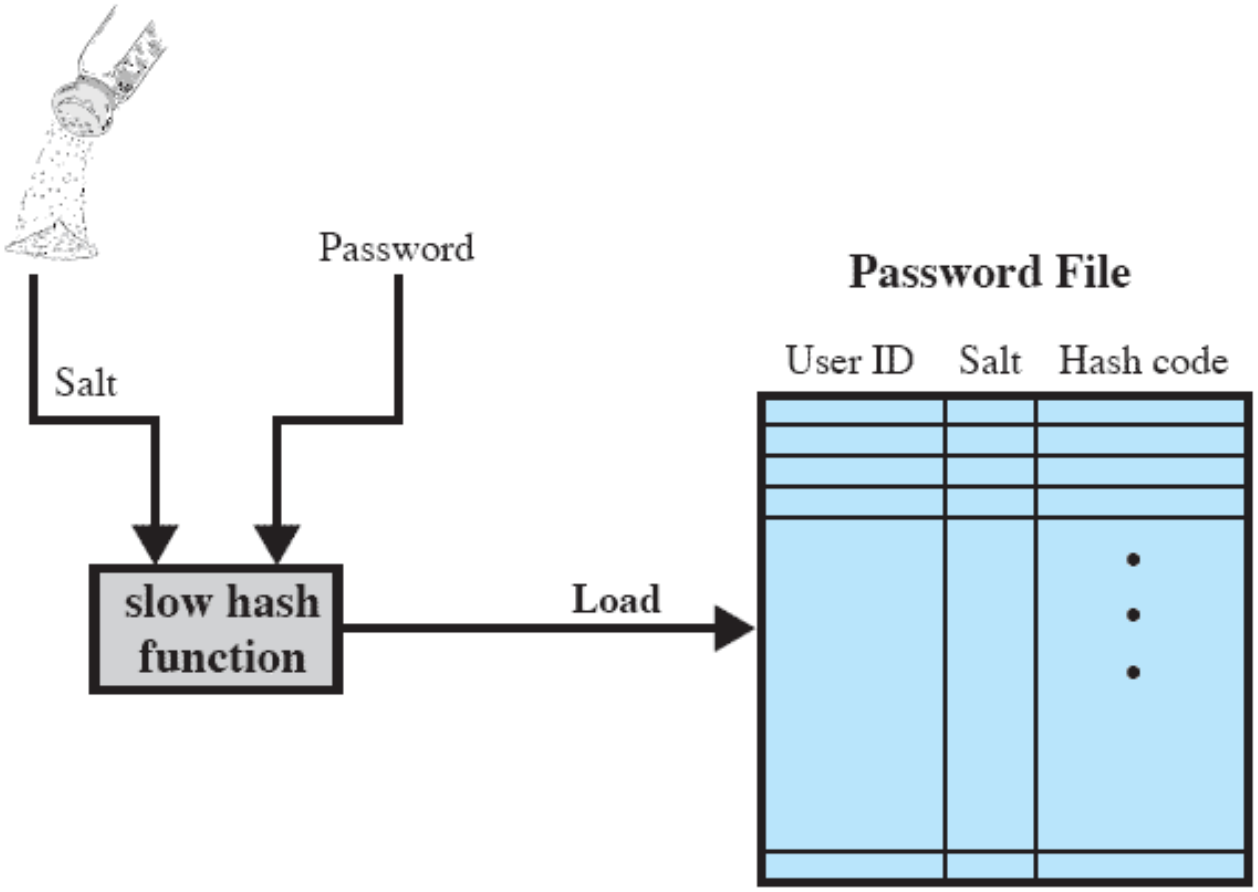
Password-Based Authentication

A widely used line of defense against intruders is the password system

The password serves to authenticate the ID of the individual logging on to the system

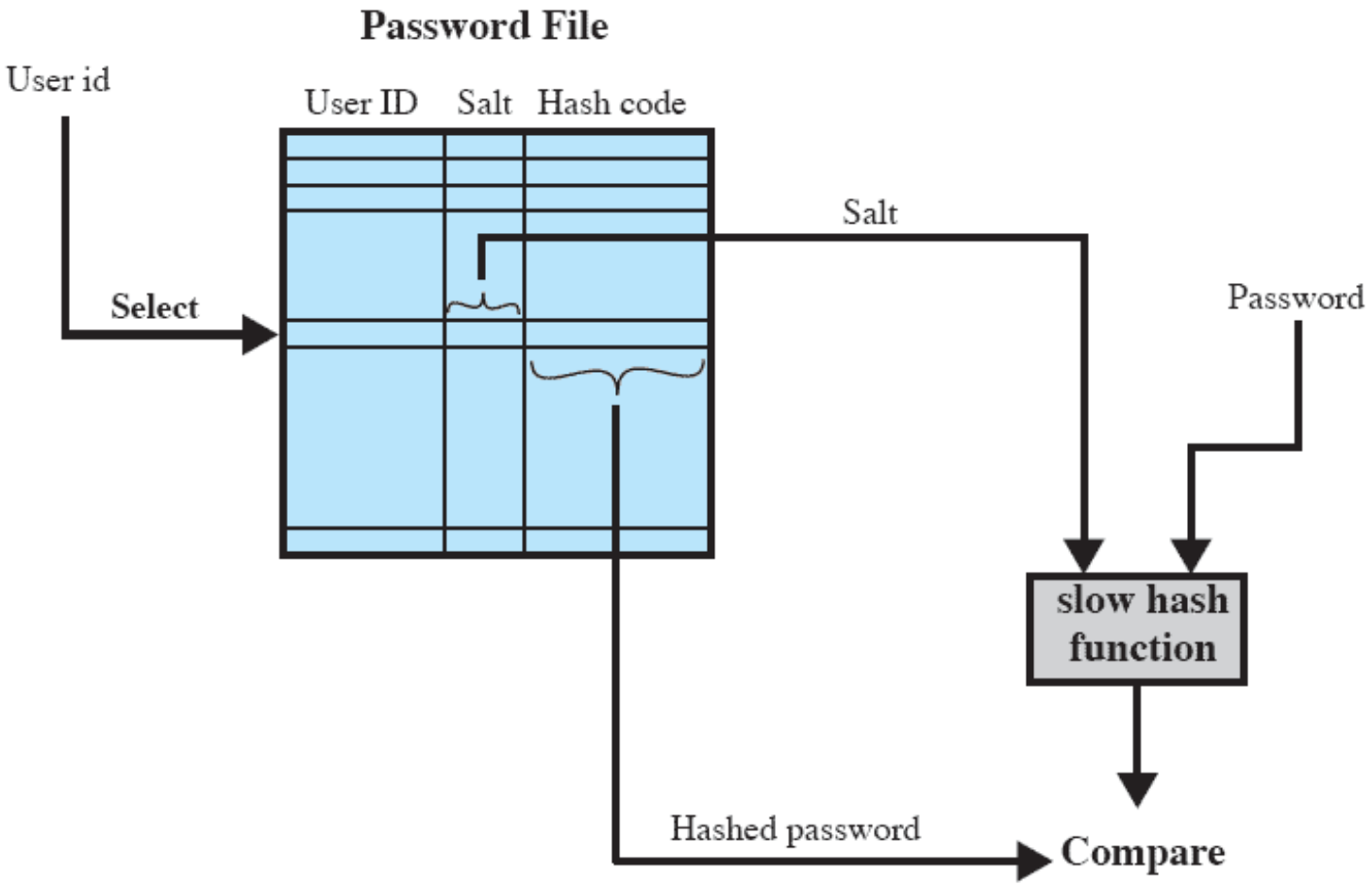
The ID provides security by:

Hashed Passwords/Salt Value



(a) Loading a new password

UNIX Password Scheme



(b) Verifying a password

Salt

Serves three purposes:

1. prevents duplicate passwords from being visible in the password file

   even if two users choose the same password, the passwords will be assigned different salt values

2. greatly increases the difficulty of offline dictionary attacks
3. it becomes nearly impossible to find out whether a person with passwords on two or more systems has used the same password on all of them
4. No precomputation of hashes possible (rainbow table)

RainbowTable

- Data structure developed by Philippe Oechslin
- Brute force needs for hashing (without Rainbow Table (RT))....
  - ...A lot of time (depending on hash length: on average O(2n /2))
  - ...Much memory, if hash values are precalculated
- RainbowTab: Time-Memory Tradeoff
- Much faster than O(2n /2) hash operations
- Substantially less memory than 2n

UNIX Password Scheme

There are two threats to the UNIX password scheme:

- a user can gain access on a machine using a guest account
- ***Password cracker*** – password guessing program

If an opponent is able to obtain a copy of the password file, a cracker program can be run on another machine at leisure

- this enables the opponent to run through millions of possible passwords in a reasonable period